

420-301-VA: Web Services

Lab 3: HTTP Fundamentals and Web Service Clients

Pravish Sainath
Computer Science & Technology
Vanier College

Introduction

In Lab 1, we learned how to transmit arbitrary text across a network using TCP Sockets. However, for a web browser (client) and a web server to understand each other, they cannot simply send random text; they must speak a standardized language. That language is the **Hypertext Transfer Protocol (HTTP)**.

HTTP is an application-layer protocol that runs *over* a TCP socket connection. In this lab, you will explore HTTP at three different levels of abstraction:

1. **Raw Sockets:** Manually writing the exact text string required by the HTTP protocol and sending it over a TCP socket.
2. **Command-Line Clients:** Using `curl` to initiate requests and inspect the raw client-server handshake.
3. **Graphical API Clients:** Using industry-standard tools like Postman or Insomnia to consume modern REST Web Services easily.

Objectives

This lab bridges the gap between raw networking and modern web services. By the end of this lab, you should be comfortable with:

- Understanding the plain-text structure of an HTTP Request (Request Line, Headers, Blank Line, Body).
- Sending a raw HTTP `GET` method over a TCP `Socket` to port 80.
- Reading and parsing the HTTP Response (Status Line, Headers, Body).
- Initiating HTTP requests from the terminal using `curl` and inspecting headers.
- Consuming public REST Web Services using graphical tools (Postman/Insomnia) and observing status codes like 200 `OK` and 201 `Created`.

Instructions

- Review the lecture notes regarding HTTP URIs, methods, and request/response formatting before starting.
- For Part 1 and 2, write the Java code in your IDE and execute it.
- For Part 3 and 4, you will use your operating system's terminal and a downloaded GUI tool.
- Copy-paste your formatted code and screenshot your terminal/GUI outputs for each exercise to validate that your requests successfully retrieved data from the live internet.
- You can work with your peers, but each person must write, execute, and document their own work.

1 Part 1: The Raw HTTP GET Request

1.1 Concept Review: HTTP Request Structure

When you type “`http://example.com/index.html`” into your browser, the browser opens a TCP socket to `example.com` on port 80 (the default HTTP port) and sends a specific text string formatted with carriage returns and line feeds (`\r\n`).

A standard HTTP Request consists of:

1. **The Request Line:** The HTTP method (`GET`), the URI path (`/index.html`), and the protocol version (`HTTP/1.1`).
2. **Headers:** Key-value pairs providing metadata (e.g., `Host: example.com`). The `Host` header is mandatory in HTTP/1.1.
3. **A Blank Line:** A single empty line (`\r\n`) to indicate the end of the headers.
4. **The Body:** (Optional, usually used for `POST` or `PUT` requests).

```
// Note the \r\n (Carriage Return + Line Feed) used by the HTTP specification
String httpRequest =
    "GET / HTTP/1.1\r\n" +
    "Host: example.com\r\n" +
    "Connection: close\r\n" +
    "\r\n"; // The mandatory blank line
```

Listing 1: Example: The exact string format of a Raw HTTP Request

Exercise 1: The Hand-Crafted Browser

In this exercise, you will act as a primitive web browser. You will open a socket to a real website, hand-craft the HTTP request string, and print the raw response returned by the server.

1. The Setup: Create a class named `RawHttpClient`.

- In the `main` method, declare a `String` variable for the host: `String host = "example.com";`
- Declare an `int` variable for the port: `int port = 80;` (Port 80 is for unencrypted HTTP).

2. The Socket and Streams:

- Create a `Socket` connecting to the host and port inside a `try-with-resources` block.
- Wrap the socket's `OutputStream` in a `PrintWriter`. Pass `true` to enable auto-flushing!
- Wrap the socket's `InputStream` in a `BufferedReader`.

3. Sending the Request:

- Use `out.print()` (not `println()`) to send the exact request string. We use `print()` so we can manually control the strict `\r\n` line endings required by HTTP servers.
- Send the request line: `out.print("GET / HTTP/1.1\r\n");`
- Send the mandatory host header: `out.print("Host: " + host + "\r\n");`
- Send a connection header telling the server to close the socket after responding: `out.print("Connection: close\r\n");`
- Send the blank line to finish the request: `out.print("\r\n");`
- Call `out.flush();` just to be safe.

4. Reading the Response:

- Create a `String line;` variable.
- Create a `while ((line = in.readLine()) != null)` loop.
- Inside the loop, simply print the line to the console using `System.out.println(line);`.

5. Observe the Results:

- Run your application. You should see a large block of text.
- Look at the very first line of the output. It should say `"HTTP/1.1 200 OK"`. This is the **Status Line**.
- Below that, you will see several **Headers** (e.g., `Content-Type`, `Content-Length`, `Date`).
- Below the headers, you will see a blank line, followed by the raw HTML code (the **Body**) of the `example.com` homepage!

2 Part 2: Exploring Status Codes and Headers

2.1 Concept Review: Parsing the HTTP Response

A web browser doesn't just print the raw text; it parses it. It reads the Status Line to know if the request succeeded (200 OK) or failed (404 Not Found). It reads the Headers to know how to interpret the body (e.g., `Content-Type: text/html` vs `application/json`).

The rule for parsing an HTTP response is simple: Read line by line. The first line is the status. Subsequent lines are headers. The headers end the moment you read a completely empty line (`""`). Everything after that empty line is the message body.

Exercise 2: The HTTP Inspector

In this exercise, you will modify your client to programmatically separate the Status Line, the Headers, and the Body, and you will request different URIs to intentionally trigger different HTTP status codes.

1. **The Setup:** Create a new class named `HttpInspector`.

- Copy the socket connection and stream wrapping logic from Exercise 1.
- We will use `example.com` on port 80 again.

2. **Triggering a 404 Not Found:**

- Instead of requesting the root path (`/`), modify your request line to ask for a file that doesn't exist:

```
out.print("GET /this-page-does-not-exist.html HTTP/1.1\r\n");
```
- Send the Host, Connection: `close`, and blank line exactly as before.

3. **Parsing the Response:**

- Instead of a simple loop, read the very first line outside of a loop:

```
String statusLine = in.readLine();
```
- Print the status line clearly:

```
System.out.println("STATUS: " + statusLine);
```
- Now, create a loop to read the headers. This loop should stop when it hits an empty string (the blank line separating headers from the body):

```
System.out.println("--- HEADERS ---");
String headerLine;
while ((headerLine = in.readLine()) != null && !headerLine.isEmpty()) {
    System.out.println(headerLine);
}
```
- Finally, create a loop to read the rest of the stream (the body).

```
System.out.println("--- BODY ---");
String bodyLine;
while ((bodyLine = in.readLine()) != null) {
    System.out.println(bodyLine);
}
```

4. Test and Observe:

- Run your application. Verify that the Status Line now reads “HTTP/1.1 404 Not Found”. Notice that the server still returned a Body (an HTML page containing an error message).

5. Triggering a Redirect (301):

- Change the host to "google.com". Change your GET request to: `out.print("GET / HTTP/1.1\r\n");` and the Host header to "google.com".
- Run the program. You should see a “HTTP/1.1 301 Moved Permanently” status code. Look at the headers—you will see a `Location` header telling the browser where it should go instead.

6. Thinking Questions:

- **Statelessness:** Your Java program runs, connects, gets the response, and then closes immediately. If you run it again, the server has no memory of you. How do real web applications bypass this statelessness? (Hint: Look at the headers sent back).
- **HTTPS (Security):** Try changing the port to 443 (the HTTPS port). Run the program. It will likely hang or return garbage characters. Why does a standard `Socket` fail on port 443?

3 Part 3: Command-Line HTTP with cURL

3.1 Concept Review: The curl Utility

Writing raw TCP sockets every time you want to make a web request is tedious. Real-world developers use tools that abstract the socket creation away but still give them fine-grained control over the HTTP protocol. The most famous of these tools is `curl` (Client URL), a command-line utility built into Windows, macOS, and Linux.

Exercise 3: Inspecting Traffic via CLI

In this exercise, you will use `curl` to perform the exact same HTTP requests you made in Java, but with much less effort.

1. **Basic Request:** Open your operating system's terminal (Command Prompt, PowerShell, or bash). Type the following command and hit Enter:

- `curl http://example.com`
- **Observation:** You will see the HTML body printed to the console. Notice that `curl` hides the headers and status codes by default, focusing only on the data payload.

2. **Fetching Headers Only (HEAD Request):** Sometimes you only want to know *if* a resource exists or when it was last modified, without downloading the entire body. We use the `-I` flag for this (which sends an HTTP HEAD method instead of GET).

- `curl -I http://example.com`
- **Observation:** You should now see the “HTTP/1.1 200 OK” status code and the headers, but no HTML body.

3. **Verbose Mode (The Handshake):** To truly see the protocol in action, we use the `-v` (verbose) flag. This reveals the underlying TCP connection and the exact text sent back and forth.

- `curl -v http://example.com`
- **Observation:**
 - Lines starting with `*` are `curl` doing background work (like DNS resolution and TCP socket binding).
 - Lines starting with `>` are the raw HTTP Request strings sent *to* the server (Notice how identical they look to your Java code in Exercise 1!).
 - Lines starting with `<` are the raw HTTP Response headers received *from* the server.

4 Part 4: Graphical API Clients (Postman / Insomnia)

4.1 Concept Review: Consuming REST Web Services

While `curl` is fantastic for quick terminal work, modern applications heavily rely on **RESTful Web Services** (APIs). These services typically exchange data in JSON (JavaScript Object Notation) format rather than HTML.

To test, debug, and document these APIs, developers use graphical API clients like **Postman** or **Insomnia**. These tools provide a beautiful UI to craft complex requests, handle authentication, and elegantly format JSON responses. Underlying the UI, however, they are doing exactly what your Java Socket did in Part 1.

Exercise 4: Interacting with a Public API

In this exercise, you will consume a free, dummy REST API called `JSONPlaceholder` using a graphical client.

1. The Setup:

- Download and install either [Postman](#) or [Insomnia](#) (or open the web version of Postman).
- Create a new “HTTP Request” tab in the tool.

2. Executing a GET Request:

- Set the HTTP Method dropdown to **GET**.
- Enter the following URI into the URL bar:
`https://jsonplaceholder.typicode.com/users`
- Click the **Send** button.
- **Observation:**
 - Look at the **Response Body** pane. The tool automatically formatted (pretty-printed) the raw JSON array of users so it is easy to read.
 - Find the **Status Code** indicator in the UI. It should display a green 200 OK.
 - Click on the **Headers** tab in the response pane. Note the **Content-Type:** `application/json; charset=utf-8`.

3. Executing a POST Request: REST APIs use different HTTP Methods to signify different actions. A POST request means “create new data”.

- Change the HTTP Method dropdown to **POST**.
- Change the URI to: `https://jsonplaceholder.typicode.com/posts`
- To send data, you must provide a Body. Click the **Body** tab, select **raw**, and change the type dropdown from Text to **JSON**.
- Paste the following JSON payload into the body editor:

```
{
  "title": "My First Post",
  "body": "This is the content of my post created via Postman.",
  "userId": 1
}
```

- Click **Send**.
- **Observation:** Look at the Status Code. It should now read **201 Created**, which is the standard HTTP status for successfully creating a resource. The response body will echo back your data along with a newly generated **"id"**.

4. **Thinking Question:**

- Based on the raw socket work you did in Part 1, what exactly did Postman send behind the scenes when you clicked “Send” for the POST request? What Headers *must* it have added automatically so the server knew it was receiving JSON data?

References

- [MDN Web Docs: An Overview of HTTP](#): The definitive, easy-to-read guide on how HTTP works, including structure, status codes, and headers.
- [MDN Web Docs: HTTP Response Status Codes](#): A comprehensive list of all status codes (2xx, 3xx, 4xx, 5xx) and their meanings.
- [cURL Official Tutorial](#): A comprehensive guide on how to use command-line options for `curl`, dealing with headers, forms, and authentication.
- [Postman Learning Center](#): The official getting-started documentation for building and sending requests in Postman.
- [RFC 7230: HTTP/1.1 Message Syntax and Routing](#): The official internet standard defining the strict string formatting (`\r\n`) required to formulate valid HTTP requests and responses.